

Úvod do databázových systémů

Kapitola 11
Jazyk SQL (4), transakce

Dotazy na „neexistující“ informaci

■ příklad

- vypiš čísla a třídy silnic a počet mostů na nich
- následující dotaz nevypíše silnice bez mostů

```
SELECT číslo_s, třída, COUNT(*)  
FROM Silnice NATURAL JOIN Leží_na  
GROUP BY číslo_s, třída;
```

- je třeba použít vnější spojení

proč nelze použít
COUNT(*) ??

```
SELECT číslo_s, třída, COUNT(číslo_m)  
FROM Silnice LEFT JOIN Leží_na USING(číslo_s)  
GROUP BY číslo_s, třída;
```

Dotazy na „neexistující“ informaci

- příklad

- jiné řešení pomocí vnořeného poddotazu

```
SELECT číslo_s, třída, (SELECT COUNT(číslo_m)
                        FROM Leží_na L
                        WHERE L.číslo_s = S.číslo_s)
FROM Silnice S;
```

Kvantifikátory $\exists$ a $\forall$

- dotazy, které vedou na kvantifikátory $\exists$ a $\forall$
 - dotazy, obsahující „všechny“, „žádný“, „každý“, ap.
 - existenční kvantifikátor $\exists$
 - v SQL implementován pomocí **[NOT] EXISTS**
 - testuje (ne)práznost výsledku vnořeného poddotazu
 - univerzální kvantifikátor $\forall$
 - nutno převést na existenční kvantifikátor (viz TZI)
 - $\forall x (P(x))$ je ekvivalentní $\neg \exists x (\neg P(x))$
 - zbývá tedy vyřešit, co s „ $\neg \exists$ “
 - často je nutné vytvořit universum (všechny možné prvky) a z něj odstranit prvky, které *mají* danou vlastnost – zbydou prvky, které vlastnost nemají
 - někdy lze použít **NOT EXISTS**

Kvantifikátory $\exists$ a $\forall$

- příklad

- najdi silnice, na nichž neleží žádný most

```
SELECT *
```

```
FROM Silnice S
```

```
WHERE NOT EXISTS
```

```
(SELECT 1
```

```
FROM Leží_na L
```

```
WHERE L.číslo_s = S.číslo_s);
```

je jedno, co dotaz
vrací, důležitá je
(ne)prázdnost

Kvantifikátory $\exists$ a $\forall$

- další příklady z přednášky o relační algebře
 - tabulky Kino, Film a Program
 - vypiš názvy kin, která nic nehrají
 - pomocí rozdílu množin

```
(SELECT název_k FROM Kino)  
EXCEPT  
(SELECT název_k FROM Program);
```

- pomocí NOT EXISTS

```
SELECT název_k FROM Kino K  
WHERE NOT EXISTS  
    (SELECT * FROM Program P  
     WHERE P.název_k = K.název_k);
```

Kvantifikátory $\exists$ a $\forall$

- vypiš názvy kin, která nic nehrají
 - pomocí NOT IN (implementace $\notin$)

```
SELECT název_k FROM Kino  
WHERE název_k NOT IN (SELECT název_k FROM Program);
```

- pomocí $\neq$

```
SELECT název_k FROM Kino  
WHERE název_k  $\neq$  (SELECT název_k FROM Program);
```

- pomocí vnějšího spojení

```
SELECT název_k  
FROM Kino LEFT JOIN Program USING(název_k)  
WHERE název_f IS NULL;
```

Kvantifikátory $\exists$ a $\forall$

- názvy kin, která nehrají žádný film s hercem Brando
 - pomocí rozdílu množin
 - od všech kin odečteme kina, které *hrají něco s Brandem*

```
(SELECT název_k FROM Kino) EXCEPT  
(SELECT název_k  
FROM Program NATURAL JOIN Film  
WHERE herec = 'Brando');
```

- výsledkem budou i kina, která nehrají vůbec nic
- názvy kin, která **hrají**, ale nehrají žádný film s Brando:

```
(SELECT název_k FROM Program) EXCEPT  
(SELECT název_k  
FROM Program NATURAL JOIN Film  
WHERE herec = 'Brando');
```


Kvantifikátory $\exists$ a $\forall$

- názvy kin, která nehrají žádný film s hercem Brando
 - pomocí NOT EXISTS

```
SELECT název_k FROM Kino K
WHERE NOT EXISTS
    (SELECT *
     FROM Program P NATURAL JOIN Film
     WHERE herec = 'Brando' AND P.název_k = K.název_k);
```

- pomocí NOT IN

```
SELECT název_k FROM Kino
WHERE název_k NOT IN
    (SELECT název_k
     FROM Program NATURAL JOIN Film
     WHERE herec = 'Brando');
```

Kvantifikátory $\exists$ a $\forall$

- názvy kin, která nehrají *některý* film s hercem Brando (tj. nehrají aspoň jeden nebo taky **žádný**)
 - lze řešit buď přepisem rel. algebry nebo počítáním

```
SELECT název_k
FROM Kino LEFT JOIN Program USING(název_k)
WHERE název_f IS NULL OR
      název_f IN (SELECT název_f FROM Film
                  WHERE herec = 'Brando')
GROUP BY název_k
HAVING COUNT(DISTINCT název_f) <
      (SELECT COUNT(DISTINCT F.název_f)
       FROM Film F
       WHERE herec = 'Brando');
```

Kvantifikátory $\exists$ a $\forall$

- názvy kin, která nehrají *některý* film s hercem Brando (tj. nehrají aspoň jeden nebo taky **žádný**)
 - lze řešit buď přepisem rel. algebry nebo počítáním

```
SELECT název_k
FROM Kino LEFT JOIN
      (SELECT Program NATURAL JOIN Film
WHERE herec = 'Brando') AS X
      USING(název_k)
GROUP BY název_k
HAVING COUNT(DISTINCT název_f) <
      (SELECT COUNT(DISTINCT F.název_f)
FROM Film F
WHERE herec = 'Brando');
```

Kvantifikátory $\exists$ a $\forall$

- názvy kin, která nehrají *některý* film s hercem Brando (tj. nehrají aspoň jeden nebo taky **žádný**)
 - jiné řešení (patrně pomalejší)

```
SELECT název_k
FROM Kino K
WHERE
  (SELECT COUNT(DISTINCT název_f)
   FROM Program P NATURAL JOIN Film
   WHERE P.název_k = K.název_k AND herec = 'Brando') <
  (SELECT COUNT(DISTINCT F.název_f)
   FROM Film F
   WHERE herec = 'Brando');
```

Kvantifikátory $\exists$ a $\forall$

- názvy kin, hrající všechny filmy s hercem Brando
 - počítáním
 - předpokládá existenci **aspoň jednoho filmu s Brandem** v databázi filmů

```
SELECT název_k
FROM Kino NATURAL JOIN Program
WHERE název_f IN
    (SELECT název_f FROM Film
     WHERE herec = 'Brando');
GROUP BY název_k
HAVING COUNT(DISTINCT název_f) =
    (SELECT COUNT(DISTINCT F.název_f)
     FROM Film F
     WHERE herec = 'Brando');
```

Kvantifikátory $\exists$ a $\forall$

- názvy kin, hrající všechny filmy s hercem Brando
 - počítáním
 - **nepředpokládá** existenci aspoň jednoho filmu s Brandem v databázi filmů

```
SELECT název_k
FROM Kino LEFT JOIN
      (SELECT Program NATURAL JOIN Film
WHERE herec = 'Brando') AS X
      USING(název_k)
GROUP BY název_k
HAVING COUNT(DISTINCT název_f) =
      (SELECT COUNT(DISTINCT F.název_f)
FROM Film F
WHERE herec = 'Brando');
```

Kvantifikátory $\exists$ a $\forall$

- názvy kin, hrající všechny filmy s hercem Brando
 - rozdíl množin a NOT EXISTS
 - testujeme prázdnotu množiny, kterou dostaneme, když od všech filmů s Brandem odečteme filmy na programu daného kina (tím nám zbydou ty, které kino nehraje)

SELECT název_k

FROM Kino K

WHERE NOT EXISTS

((SELECT DISTINCT jmén

FROM Program NATURAL JOIN Film

WHERE herec = 'Brando')

EXCEPT

(SELECT DISTINCT jméno_f

FROM Program P

WHERE P.název_k = K.název_k));

záleží, co myslíme tím
všechny: všechny na
programu nebo všechny v
seznamu filmů

Kvantifikátory $\exists$ a $\forall$

- názvy kin, nehrající *některý* film s hercem Brando
 - rozdíl množin a EXISTS
 - testujeme **ne**prázdnost množiny, kterou dostaneme, když od všech filmů s Brandem odečteme filmy na programu daného kina (tím nám zbydou ty, které kino nehraje)

```
SELECT název_k
FROM Kino K
WHERE EXISTS
    ((SELECT DISTINCT jméno_f
     FROM Program NATURAL JOIN Film
     WHERE herec = 'Brando')
     EXCEPT
    (SELECT DISTINCT jméno_f
     FROM Program P
     WHERE P.název_k = K.název_k));
```


Kvantifikátory $\exists$ a $\forall$

- názvy kin, která hrají všechny filmy z programu
 - počítáním

```
SELECT název_k
FROM Kino LEFT JOIN Program USING(název_k)
GROUP BY název_k
HAVING COUNT(DISTINCT název_f) =
    (SELECT COUNT(DISTINCT P.název_f)
     FROM Program P);
```

Kvantifikátory $\exists$ a $\forall$

- názvy kin, která hrají všechny filmy z programu
 - rozdíl množin a NOT EXISTS

```
SELECT název_k
FROM Kino K
WHERE NOT EXISTS
    ((SELECT DISTINCT jméno_f FROM Program)
     EXCEPT
     (SELECT DISTINCT jméno_f
      FROM Program P
      WHERE P.název_k = K.název_k));
```

Konstrukt CASE

- CASE – běžné podmíněné větvení
 - pro transformaci typu hodnot apod.

```
SELECT a,  
       CASE WHEN a=1 THEN 'one'  
            WHEN a=2 THEN 'two'  
            ELSE 'other'  
       END  
FROM test;
```

```
SELECT a,  
       CASE a WHEN 1 THEN 'one'  
            WHEN 2 THEN 'two'  
            ELSE 'other'  
       END  
FROM test;
```

Konstrukt CASE

- CASE – běžné podmíněné větvení
 - pro zajištění IO
 - chceme zajistit, aby každou silnici první třídy vlastnil stát

```
CREATE TABLE Silnice (  
    číslo_s INT PRIMARY KEY,  
    třída INT NOT NULL CHECK (třída>0)  
    vlastník VARCHAR(50) NOT NULL,  
    CONSTRAINT chk_vlastník CHECK (  
        CASE WHEN třída=1 THEN vlastník='stát' END )  
);
```

Konstrukt COALESCE

■ COALESCE

- vybere ze seznamu v závorkách první hodnotu, která není NULL

COALESCE(A, B, C, D);

- řešení případů, kdy očekáváme NULL
 - například u agregačních funkcí

COALESCE(Vypujcky.Cena, "Cena neni urcena");

- příklad na pokoje a hosty: vypiš příjmení všech hostů a u těch, kteří někde bydlí nebo bydleli, vypiš číslo pokoje, u neubytovaných nulu

```
SELECT příjmení, COALESCE(číslo, 0)
FROM BYDLÍ RIGHT NATURAL JOIN HOST;
```

Konstrukt COALESCE

■ COALESCE

- příklad: chceme vypsat čísla silnic, počet mostů na každé silnici (vč. nul) a nejmenší nosnost mostu na každé silnici

```
SELECT číslo_s,  
       COUNT(číslo_m),  
       MIN(COALESCE(nosnost, 100000000))  
FROM   Silnice LEFT NATURAL JOIN  
       (Leží_na NATURAL JOIN Most)  
GROUP BY číslo_s;
```

Řádkové výrazy

- výraz

$$(R.cena, R.datum) = (S.cena, S.datum)$$

Ize použít místo

$$R.cena = S.cena \text{ AND } (R.datum = S.datum)$$

- výraz

$$(R.cena, R.datum) > (S.cena, S.datum)$$

Ize použít místo

$$R.cena > S.cena \text{ OR } (R.cena = S.cena \text{ AND } R.datum > S.datum)$$

Transakce

- koncept, jak zajistit
 - paralelní přístup k datům
 - zotavení z chyb
- operace se vykonávají v rámci nedělitelných programových jednotek – transakcí
- transakce
 - nedělitelná, buď se provede celá nebo vůbec
 - převádí DB z jednoho konzistentního stavu do jiného konzistentního stavu
 - dokud není potvrzena, dílčí změny nejsou viditelné pro ostatní transakce
 - efekty úspěšné transakce jsou již trvalé

Transakce

■ transakce

- posloupnost akcí (čtení, zápis, výpočet...), s nimiž se zachází jako s jedním celkem
- typický úkol – převod peněz z účtu na účet
 - odečíst peníze ze zdrojového účtu
 - přičíst peníze na cílový účet
 - musí proběhnout buď obě dílčí operace nebo žádná

■ řízení transakcí

- převážně automatické
 - pro každý SQL dotaz zavádí DBMS samost. transakci
- explicitně lze transakce řídit pomocí příkazů **COMMIT** a **ROLLBACK**

Transakce

- stav transakce
 - aktivní – počátek provádění transakce
 - částečně potvrzený – těsně před COMMIT
 - chybný – vyskytla se chyba
 - zrušený – po ROLLBACKu
 - potvrzený – po COMMITu

Transakce

- implementace transakcí
 - je nutné vést tzv. žurnál (log file)
 - zapisuje se, co se provádí
 - jelikož transakce jsou používány automaticky při jakékoli manipulaci s daty v databázi, každý příkaz je žurnálován
 - 1. odložená realizace změn
 - do žurnálu se píší operace, ale nejsou prováděny hned
 - pokud všechny simulované transakce dopadnou dobře a jsou potvrzeny, změny se stanou viditelnými a mohou být zapsány do DB
 - téměř nikdy tak není nutný ROLLBACK
 - 2. bezprostřední realizace změn
 - vše se zapisuje do DB rovnou, ale žurnál změny detailně eviduje, aby se dal provést ROLLBACK

Transakce

- implementace transakcí
 - DBMS používají obvykle odloženou realizaci změn (v Postgresu WAL – write-ahead logging)
 - změny vyvolané transakcí se mohou stát viditelnými už ve chvíli, kdy je na disk zapsán pouze log-file,
 - změna vlastních dat. souborů může být dále odložena
 - dotaz na DB v tomto stavu vyžaduje kombinaci informací v datových souborech a v žurnálu
 - původ tkví v pomalosti starých diskových systémů, kdy bylo podstatně rychlejší zapsat (a číst) log-file, než zapisovat celý rozsáhlý či fragmentovaný datový soubor
 - chceme-li vynutit fyzické zapsání změn z žurnálu do datových souborů, je nutno použít samostatný příkaz CHECKPOINT
 - v praxi není obvykle nutné, vhodné je to např. před zálohováním databáze

Transakce

- paralelní zpracování
 - není efektivní všechny transakce provádět za sebou, proto je snaha je vzájemně prokládat
 - vzniká závislost transakcí jedna na druhé
 - pozor, transakce nevidí změny, které právě provádí jiná transakce
 - některé typy operací nelze libovolně prokládat
 - např. čtení a zápis téhož objektu
 - nutnost dodržení serializovatelnosti
 - takové proložení operací, které vede ke stejnému výsledku, jako kdyby byly transakce prováděny za sebou
 - lze řešit algoritmicky, ale je to náročné
 - konfliktní operace: typu READ a WRITE

Transakce

- paralelní zpracování
 - neošetřené prokládání transakcí
 - typické problémy
 - **lost update** – T1 a T2 po sobě zapíší různé hodnoty do téhož objektu, efekt první T1 je ztracen
 - **dirty read** – T1 zapíše hodnotu do objektu, T2 si tuto zapsanou hodnotu přečte, načež T1 provede ROLLBACK; hodnota přečtená transakcí T2 tedy nebyla definitivní
 - **unrepeatable read** – T1 přečte hodnotu, T2 tuto hodnotu změní => další čtení této hodnoty transakcí T1 vrátí jiný výsledek než poprvé
 - **phantom problem** – T1 čte data z tabulky (např. SELECT), mezitím do této tabulky T2 vloží nové řádky; T1 při dalším provedení téhož dotazu vrátí jiná data.
 - speciální případ neopakovatelného čtení

Transakce

- paralelní zpracování
 - řešení konfliktů
 - existují různé strategie na omezování výše uvedených nežádoucích jevů
 - při řízení transakcí lze nastavit tzv. isolation level
 - stupeň izolace, určuje, které z problémů budou eliminovány.
 - čím vyšší stupeň izolace, tím lepší ochrana před anomáliemi, ovšem tím horší prostupnost databáze pro víceuživatelské operace
 - typický způsob ošetření konfliktů je **uzamykání objektů**

Transakce

- paralelní zpracování
 - uzamykání objektů
 - je možné na různé úrovni detailu (granularity)
 - nejhrubější je uzamknout celý systém, což je ekvivalentem sériového provádění transakcí
 - jemnější je zamykání tabulek
 - nejjemnější je zamykání řádků v tabulkách (record locking)
 - jemné uzamykání je náročnější a alokuje velké množství zdrojů
 - někdy databázový stroj přistupuje k tzv. eskalaci zámků, kdy snižuje granularitu zamykání tak, že např. namísto mnoha zamčených řádků jedné tabulky zamkne celou tuto tabulku

Transakce

- paralelní zpracování
 - uzamykání objektů
 - jednoduchý uzamykací protokol – principy
 - transakce zamkne objekt, chce-li jej používat
 - transakce nezamyká již zamčený objekt
 - transakce může odemknout jen to, co sama zamkla
 - po ukončení transakce jsou všechny objekty odemknuty
 - dvoufázový protokol
 - v první fázi se postupně zamykají potřebné objekty, ale nic se neodemyká
 - v druhé fázi se objekty postupně odemykají, ale žádný se již nesmí zamknout
 - zajišťuje uspořádatelnost
 - **velké nebezpečí deadlocku!**

Transakce

- zotavení z chyb
 - transakce je i jednotkou zotavení z chyb
 - pokud dojde v průběhu transakce k chybě, vrátí se DB do stavu před transakcí

Transakce v PostgreSQL

- příkazy `BEGIN`, `COMMIT`, `ROLLBACK`, `SAVEPOINT`
- automatické i uživatelsky řízené zamykání
- např.

`BEGIN;`

`UPDATE accounts SET balance = balance + 100.00`

`WHERE name = 'Bob';`

`UPDATE accounts SET balance = balance - 100.00`

`WHERE name = 'Alice';`

`COMMIT;`

Transakce v PostgreSQL

- řeší i problém současného vložení řádků do dvou tabulek při povinném členství
- v tabulce lze u cizího klíče nastavit modifikátor **DEFERRABLE**, který zajistí, že kontrola referenční integrity může proběhnout až na konci transakce

`fk integer references tbl2(pk) deferrable initially
deferred,`

`fk2 integer references tbl3(pk) deferrable,`

`....`

- v každé transakci nastává kontrola ref. i initially immediate
jak je nastaveno výchozí chování, lze pro aktuální transakci změnit příkazem

`SET CONSTRAINTS name DEFERRED`